

Vibe Coding Principle and Practice

杨振 助理教授

Email: zhenyang@sdu.edu.cn

MCP

本文开源了MCP协议作为连接AI系统与数据源的通用开放标准，推出了配套的SDK、Claude桌面应用的本地MCP服务器支持，以及针对Google Drive、Slack、GitHub等系统的预构建MCP服务器，推动整个生态系统的建立与规模化应用。

Introducing the Model Context Protocol – Anthropic 2024.11.25

Why

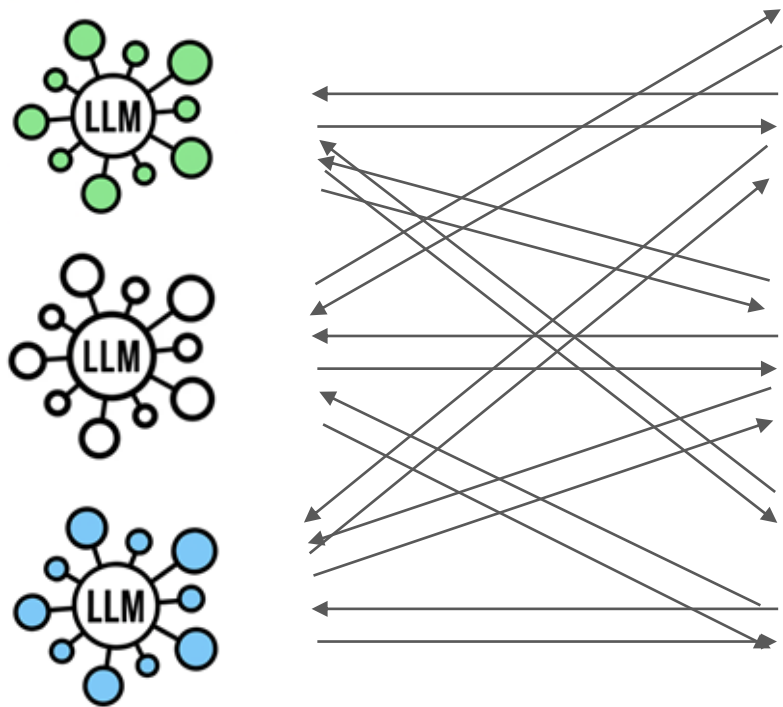
- LLM知识静态，仅随重新训练而更新
- 需要动态数据：天气、总统是谁、比特币价格、最新广告旁白等
- 当前最佳实践：**RAG** 和 **tool-calling**
- 劣势：
 - **RAG** : 1) 只读且无法执行动作-只能获取静态文本片段，无法触发外部系统的写操作、**API** 调用或实时计算（比如无法真的去买一张机票或发送一条消息）。2) 不同**RAG**无统一调用标准
 - **Tool-calling** : 可以执行动作但->1) 各个工具无统一调用标准。2) 不同模型输出的工具调用请求的格式不同

Basics - 什么是 MCP ?

- **Model Context Protocol** (模型上下文协议)
 - 开放协议，允许系统以通用方式向AI模型提供上下文
 - 历史：2024年11月之前由AI 公司 Anthropic引入

MCP的全称是模型上下文协议。它是一个开放标准，规定了如何将外部工具（如数据库、API、本地文件等）的描述和调用方式统一地提供给大语言模型。通俗地讲，MCP给“工具调用”定义了一套通用的格式和交互流程。它强调让LLM主动触发工具，而不是被动响应。MCP于2024年11月左右首次发布，这在整个AI agent演进中算是一个里程碑。

Many LLMs to many Tools



如果没有统一协议，你需要为每一个LLM应用和每一个工具之间编写专用连接器——这就是 $M \times N$ 的集成复杂度。

一方面，不同的LLM输出的工具调用格式不同，需要针对性的解析器做正则匹配。

另一方面，不同工具的调用方式、认证方式、传输协议等不同，需要针对性地设计协议来接收LLM端的请求。

所以，仅仅基于function calling，需要单独开发 $N \times M$ 次来实现 N 个LLM应用和 M 个工具的对接。

Basics

- MCP最核心的两点贡献
 - 对 LLM 侧：MCP 客户端将任意 LLM 私有的工具调用输出（如 OpenAI、Anthropic 各自的格式）统一转换成 JSON-RPC 2.0 格式的请求。
 - 对工具侧：每个工具只需封装成一个 MCP 服务器，按标准协议接收请求，并将认证、参数映射、错误处理等细节封装在内部，返回统一的JSON-RPC 2.0格式的响应。
 - 这两点共同实现了 集成复杂度从 $M \times N$ 降为 $M + N$ 的目标，这也是 MCP 作为“标准化协议”最核心的价值。

User Prompt

北京今天天气如何？东京飞纽约的航班有吗？

OpenAI 原生的工具调用输出（简化）

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "arguments": "{\"city\": \"北京\"}"
  },
},
{
  "type": "function",
  "function": {
    "name": "search_flights",
    "arguments": "{\"origin\": \"东京\", \"destination\": \"纽约\"}"
  },
}
}]
```

MCP 客户端将其转换成统一的 JSON-RPC 请求

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": { "city": "北京" }
  },
  "id": 1
},
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "search_flights",
    "arguments": { "origin": "东京", "destination": "纽约" }
  },
  "id": 2
}
```

LLM 侧

同理，Anthropic 的客户端将 Claude 的 tool_use 块转成同样格式。所有 LLM 的差异被隔离在各自客户端内部。

原生调用方式

<https://api.weather.com/v1?q=北京&key=xxx>

对 MCP 客户端只暴露统一的 JSON-RPC 格式的调用接口

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": { "city": "北京", "key": "xxx" },
    "id": 1
  }
}
```

基于 JSON-RPC 返回统一格式的响应

```
{
  "jsonrpc": "2.0",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "北京今天晴，气温 25° C，湿度 40%"
      }
    ],
    "isError": false
  },
  "id": 1
}
```

工具侧

每个工具只需封装成一个 MCP 服务器，监听 JSON-RPC 请求，内部处理认证、参数映射、错误等，返回统一响应。

Agent 需要调用工具时，会强制要求输出 json（通过训练、约束解码等技术实现）

MCP A Bit Deeper

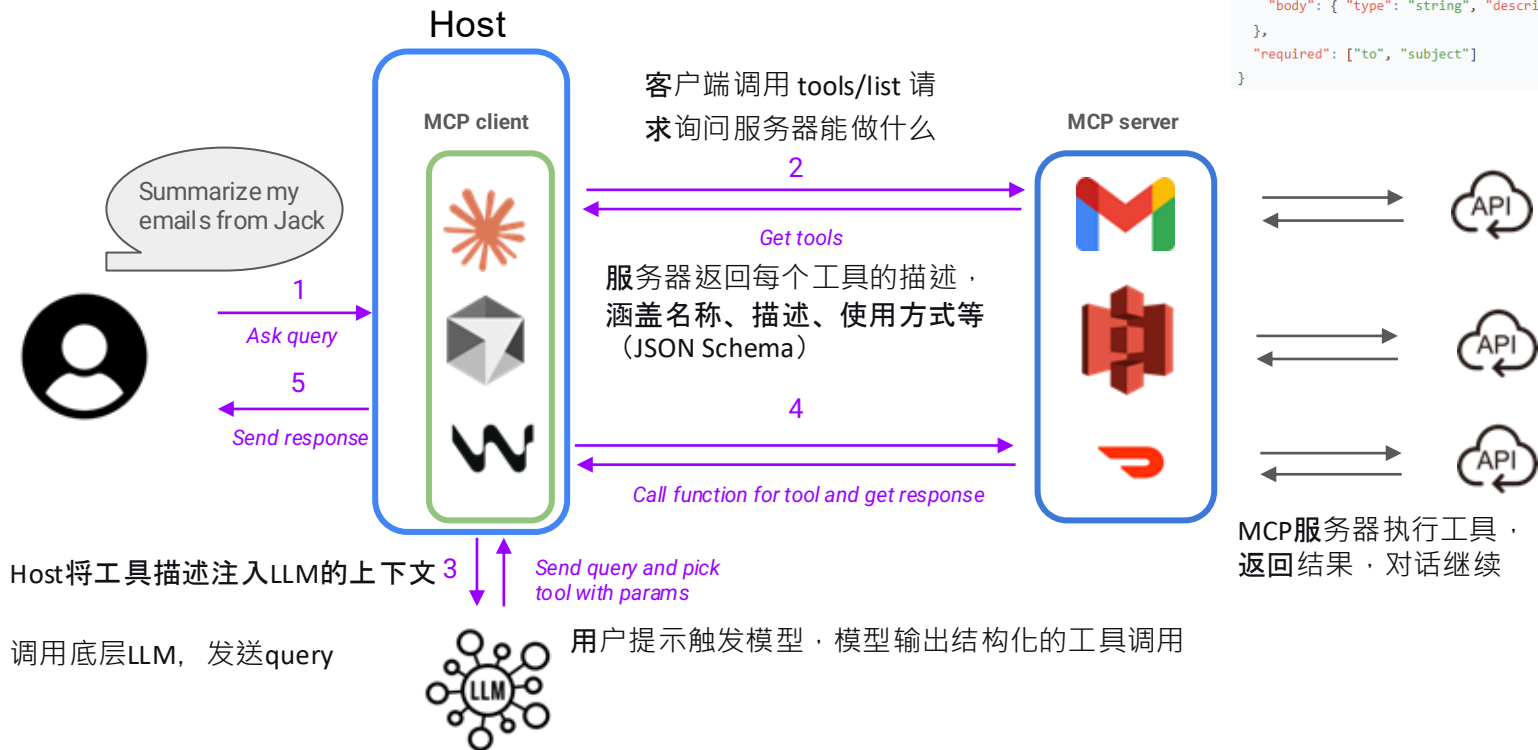
- Terminology

- **Host** : LLM应用，如 Cursor、Claude Desktop
- **MCP Client** : 嵌入在Host中的库，与每个服务器保持会话。对于WorkBuddy来说，已经内置MCP Client，可以通过`~/.workbuddy/mcp.json`文件配置MCP Server。也存在一些构建Agent的框架，如LangChain，可以通过`langchain-mcp-adapters`库实例化MCP Client，并配置可以访问的MCP Server。
- **MCP Server** : 轻量级包装器，暴露一个或多个工具，例如“获取邮件摘要”函数。如FastMCP是一个用于构建MCP Server的Python框架。
- **Tool** : 可调用的函数（数据源、API等）

MCP A Bit Deeper

JSON Schema

```
{  
  "type": "object",  
  "properties": {  
    "to": { "type": "string", "description": "收件人邮箱" },  
    "subject": { "type": "string", "description": "邮件主题" },  
    "body": { "type": "string", "description": "邮件正文" }  
  },  
  "required": ["to", "subject"]  
}
```



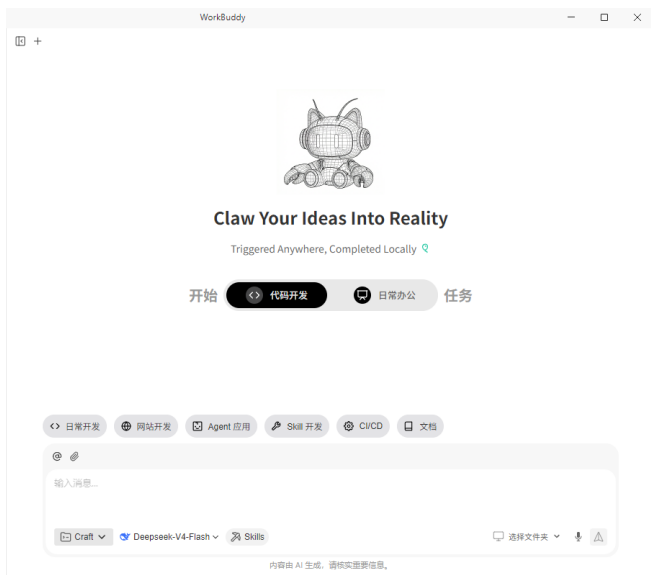
Limitations

- 当MCP服务器暴露的工具数量很多（比如几十个）时，现有的LLM Agent难以准确选择合适的工具——模型容易混淆或漏选。
- 每个工具的描述（名称、参数schema、示例）会占用宝贵的上下文窗口，如果工具太多，可能挤占用户对话空间。

在WorkBuddy中自定义MCP工具—代码质量分析器

- 需求分析

- 分析所有代码文件的行数、函数信息、函数复杂度等信息，并做对比分析。
- 需要的工具：扫描全部文件、单文件代码分析、对比分析工具



WorkBuddy 的定位是 "全场景职场 AI 智能体桌面工作台"——它是腾讯云代码助手 (CodeBuddy) 团队推出的一款面向办公自动化的桌面应用，主打的是自然语言驱动的办公任务执行

FastMCP: 构建MCP应用的一个标准框架

```
20 from mcp.server.fastmcp import FastMCP
21
22 # =====
23 # 创建 MCP Server 实例
24 # =====
25 mcp = FastMCP(
26     "code-analyzer",
27     instructions=(
28         # 对该MCP server做介绍
29         "代码质量分析工具 - 分析代码文件的行数、复杂度、函数结构、依赖导入等指标。"
30         "支持 Python、JavaScript/TypeScript、Java、C++ 等多种语言。"
31         "当你需要了解某个代码文件的质量状况、扫描项目目录结构、或对比两个文件的代码差异时使用。"
32     )
33 )
34
35 # =====
36 # 入口
37 # =====
38 if __name__ == "__main__":
39     mcp.run()
```

在code_analyzer_server.py文件中创建MCP Server实例，通过mcp.run()确定本地工具还是远程工具，通信协议不同

```
{
  "mcpServers": {
    "code-analyzer": {
      "command": "python",
      "args": ["D:\\Projects\\test_mcp\\code_analyzer_server.py"],
      "env": {}
    }
  }
}
```

mcp.json

WorkBuddy 启动时扫描这个mcp.json文件，看到 code-analyzer 条目，就会执行 code_analyzer_server.py 启动一个子进程，启动MCP Server。

```
36 # =====
37 # 工具 1: analyze_file - 分析单个代码文件
38 # =====
39 @mcp.tool(
40     description="分析单个代码文件的质量指标，支持 lines [行数统计]、functions [函数结构]、"
41     "complexity [圈复杂度估算]、imports [依赖分析]、summary [总览]。"
42     "传入 'all' 分析所有指标，或传逗号分隔的指标名，如 'lines,functions'"
43 )
44 def analyze_file(
45     file_path: str,
46     metrics: str = "all"
47 ) -> str:
48     path = Path(file_path)
49
50     # --- 文件存在性检查 ---
51     if not path.exists():
52         return json.dumps({"error": f"文件不存在: {file_path}"}, ensure_ascii=False)
53
54     if not path.is_file():
55         return json.dumps({"error": f"路径不是文件: {file_path}"}, ensure_ascii=False)
56
57     # --- 读取文件 ---
58     try:
59         content = path.read_text(encoding="utf-8")
60     except UnicodeDecodeError:
61         try:
```

通过装饰器注册工具，mcp.run()启动后，当Agent调用tools/list 请求时，自动转换为JSON格式，方便LLM阅读

```
{
  "name": "analyze_file",
  "description": "分析单个代码文件...",
  "inputSchema": {
    "properties": {
      "file_path": {
        "type": "string",
        "description": "文件路径"
      },
      "metrics": {
        "type": "string",
        "default": "all",
        "description": "要分析的指标，如 lines, functions, complexity, imports, summary"
      }
    },
    "required": ["file_path"]
  }
}
```

使用效果

帮我分析一下samples目录下的代码文件，并把原始json结果写入当目录下的report.md



WorkBuddy

10 个工具调用, 12 条过程消息

深度思考

已列出 d:\Projects\test_mcp\samples >

深度思考

获取 MCP Server 详细信息: code-analyzer

analyze_file	code-analyzer	已获取
scan_directory	code-analyzer	已获取

深度思考

> code-analyzer (scan_directory)

运行成功

深度思考

> code-analyzer (analyze_file)

运行成功

> code-analyzer (analyze_file)

运行成功

深度思考

两个文件分析完成。现在将原始结果写入 report.md。

创建 d:\Projects\test_mcp\samples\report.md +326 -0



```
321 ## 4. 快速摘要
322
323 | 文件 | 扩展名 | 大小 | 总行数 | 代码行 | 注释行 | 空白行 | 注释率 | 总圈复杂度 |
324 |-----|-----|-----|-----|-----|-----|-----|-----|-----|
325 | sample_code.py | .py | 6.4 KB | 219 | 157 | 19 | 43 | 8.7% | 50 |
326 | sample_code.js | .js | 5.7 KB | 229 | 147 | 48 | 34 | 21.0% | 56 |
327
```

LLM 负责：

- 1.理解意图 — 用户想要什么
- 2.规划顺序 — 先干什么后干什么
- 3.组合调用 — 按需调用不同的工具
- 4.后处理 — 把结果组织成用户需要的输出

Agent Skills

Agent Skills设计了一套基于文件夹的标准化格式（以SKILL.md为核心）与**渐进式披露**的加载机制，先将精简的**技能描述**载入上下文进行智能匹配，再于需要时按需调用完整指令与脚本，从而大幅提升上下文利用效率并降低运行成本。

- Skill 是一个语义完整的能力单元，它封装了一组相关的 **MCP** 工具、调用逻辑 和触发条件。对 LLM 而言，Skill 像一个“可调用的高级函数”，而非零散的底层工具。如下是WorkBuddy的marketplace中的一个案例：ArXiv论文追踪。

```
1 ---
2 name: arxiv-watcher
3 description: Search and summarize papers from ArXiv. Use when the user asks for the latest research, specific topics on ArXiv, or a daily summary of AI papers
4 description_zh: "搜索和总结 ArXiv 最新研究论文"
5 description_en: "Search and summarize the latest research papers from ArXiv"
6 version: 1.0.0
7 ---
8
9 # ArXiv Watcher
10
11 This skill interacts with the ArXiv API to find and summarize the latest research papers.
12
13 ## Capabilities
14
15 - **Search**: Find papers by keyword, author, or category.
16 - **Summarize**: Fetch the abstract and provide a concise summary.
17 - **Save to Memory**: Automatically record summarized papers to 'memory/RESEARCH_LOG.md' for long-term tracking.
18 - **Deep Dive**: Use 'web_fetch' on the PDF link to extract more details if requested.
19
20 ## Workflow 调用逻辑
21
22 1. Use 'scripts/search_arxiv.sh "<query>"' to get the XML results.
23 2. Parse the XML (look for '<entry>', '<title>', '<summary>', and '<link title="pdf">').
24 3. Present the findings to the user.
25 4. **MANDATORY**: Append the title, authors, date, and summary of any paper discussed to 'memory/RESEARCH_LOG.md'. Use the format:
26 ```markdown
27 ### [YYYY-MM-DD] TITLE_OF_PAPER
28 - **Authors**: Author List
29 - **Link**: ArXiv Link
30 - **Summary**: Brief summary of the paper and its relevance.
31 ```
```

SKILL.md

YAML格式，依赖 Agent 编排器在上下文注入时提供的技能清单



当前可用的技能列表

- arxiv-watcher: Search and summarize papers from ArXiv. Use when the user asks for the latest research, specific topics on ArXiv, or a daily summary of AI papers.
- pdf-analyzer: Extract text and tables from PDF files. Use when user uploads a PDF.
- code-reviewer: Analyze Python code for bugs and style issues. Use when user asks to review code.

Agent 编排器会根据LLM选择的Skill（以Json形式输出）加载相应Skill后续的内容，进而LLM可以知道如何进一步操作依旧调用需要的工具

```
~/workbuddy/skills/ArXiv论文追踪/
├── SKILL.md                # 技能主文件——定义能力、工作流和使用说明
├── _skillhub_meta.json     # 元数据（名称、版本、安装来源、时间戳）
├── scripts/
│   └── search_arxiv.sh     # 调用 ArXiv API (export.arxiv.org) 查询论文的 bash 脚本
├── memory/
│   └── RESEARCH_LOG.md     # 长期记忆存储
│                           # 已追踪论文的归档记录（标题、作者、链接、摘要）
```

ArXiv论文追踪Skill的目录结构

● Skills解决了哪些问题？

- 工具选择困难：LLM 只需从相对少量 Skills（例如 5~10 个）中选择，而非从几十、几百个工具中选择。因为Skill实现了分层决策：先选 Skill → Skill 内部再选工具（可由更小的模型或确定性代码完成）
- 上下文挤占问题：初始只加载 Skills 的简要列表（每个 Skill 一行描述，约 50 token），选中后根据需要动态加载内部工具详情。



Harness Engineering

2025年9月，一个仅有3-7人的团队，在5个月内用Codex Agent生成了超过100万行生产级代码，且没有一行是人类手写的。在这个过程中，团队构建了渐进式披露的上下文、刚性约束的代码审查、基于运行结果的反馈闭环等工程化体系，构建了Harness Engineering的雏形。

Agent = Model + Harness

- 模型是无状态的、没有工具、无上下文、无记忆。
- **Harness** 是除此之外其他东西，可以把一个无状态的语言模型，变成一个能持续产出工作的编码 **Agent**。
- 一个类比：**Harness**是操作系统（**OS**）；模型是 **CPU**。没有 **OS** —— 没有内存管理、没有 I/O、没有进程隔离、没有调度 —— 一颗 **CPU** 没办法和文件、网络、用户发生有用的交互。**Harness** 就是这一层 **OS**，它给模型一个结构化的运行环境，并负责模型和外部世界之间每一次交互的中介。
- **Claude Code** 是一个 **Harness**。**Trae** 是一个 **Harness**。**Cursor** 是一个 **Harness**。它们包的是不同的模型，或者是同一个模型在不同配置下的样子，但在结构上做的都是同一件事：把模型的原始能力，变成可以应用到真实工作上的形态。

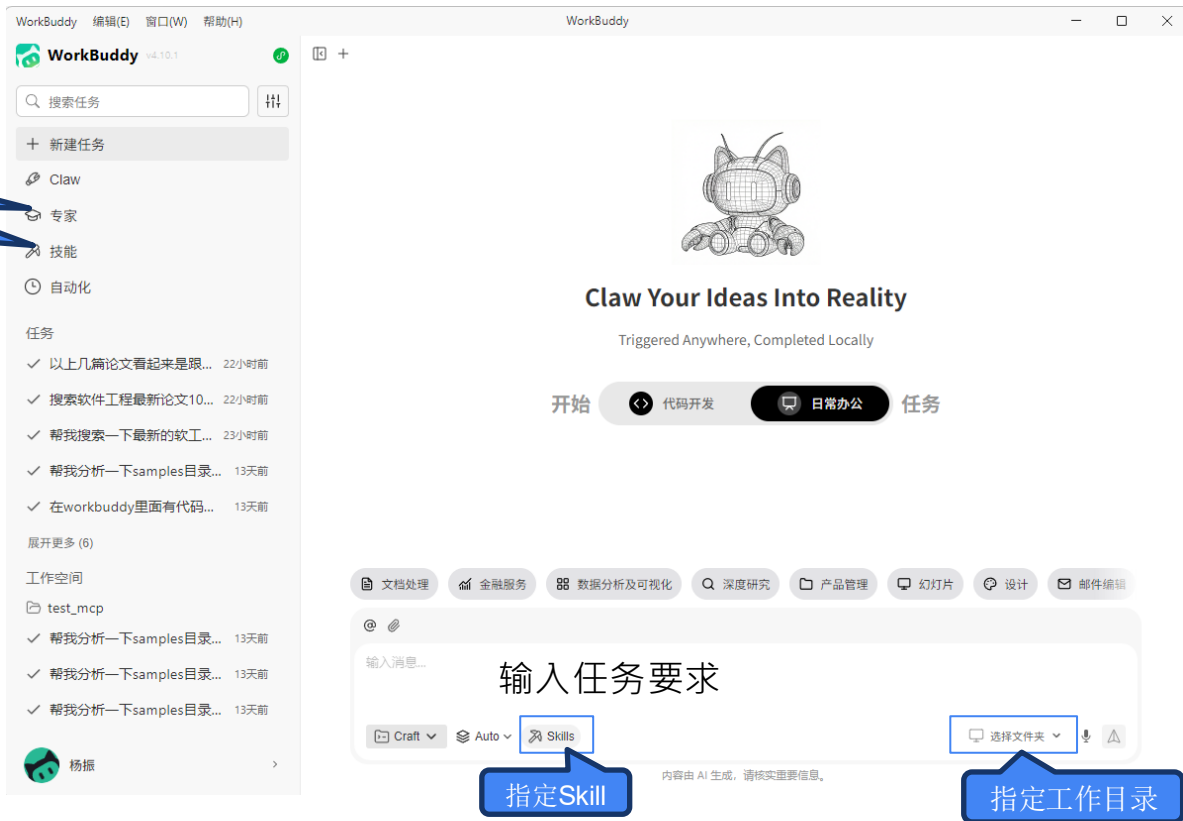
Agent Harness 的七大组件（并无定论）

- 上下文加载
 - 在模型对任何任务采取行动之前，**Harness** 会先注入上下文：项目规则、架构约束、 workflows 指令、工具相关的指引。**Claude Code** 中的 **CLAUDE.md** 就是这一层的入口。
- 工具层
 - **MCP** 工具集合
- 编排层
 - 对复杂任务进行拆分和子agent任务派发。
- 权限层
 - 确定运行边界。可以写入哪些目录、可以执行哪些命令、能不能访问网络。
- 执行钩子
 - 在模型发起动作之前或之后运行的确定性代码。如拦截一次文件写入，按规定格式输出，在失败时阻止该动作，并向模型返回一个结构化错误。
- 记忆与状态管理
 - 哪些状态会被带到下一次，上下文变得过大时如何压缩，解决记忆容量的问题。
- 会话生命周期
 - 任务从哪里开始，到哪里结束，中断后如何继续。

AI Agent产品 - WorkBuddy

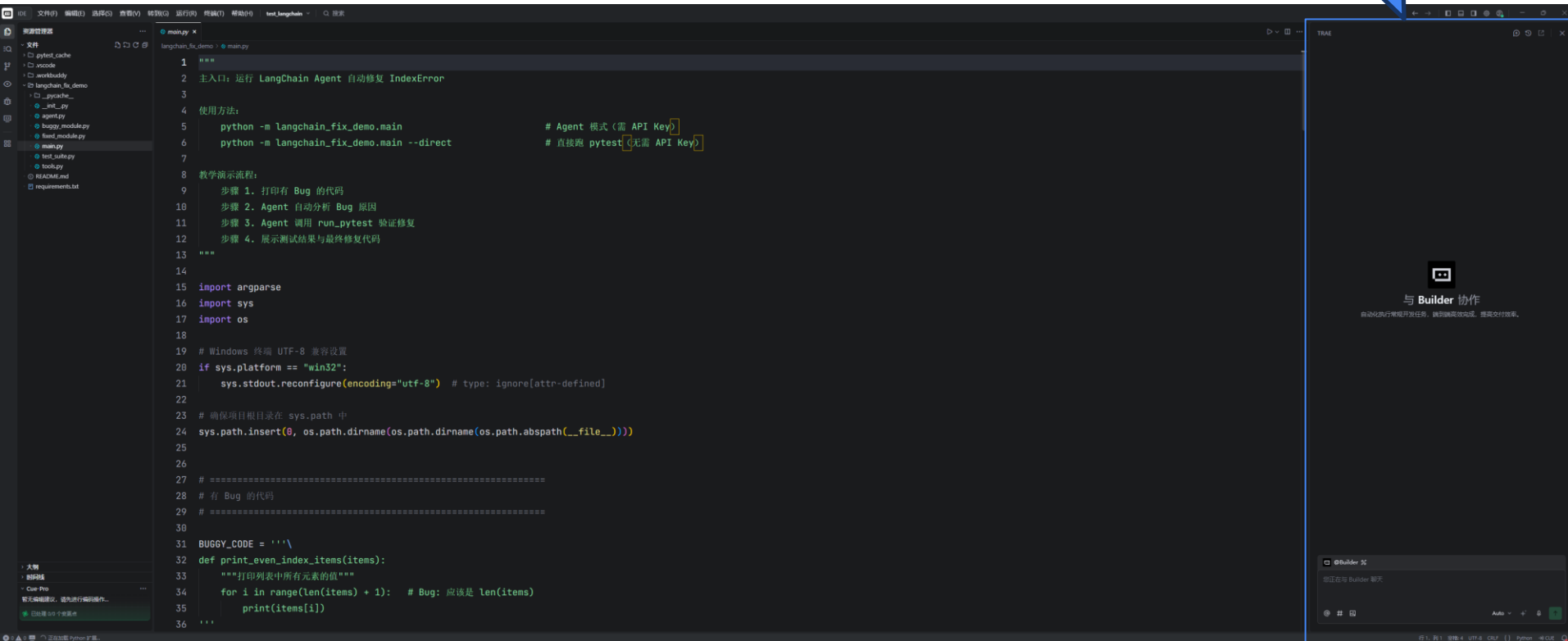
角色扮演

Skill商场



AI Agent产品 - Trae

跟Agent发布任务的窗口



```

30  ✱ 编辑 Ctrl+I 添加到对话 Ctrl+U ↵
31  BUGGY_CODE = '''\
32  def print_even_index_items(items):
33      """打印列表中所有元素的值"""
34      for i in range(len(items)):
35          print(items[i + 1]) # Bug: 应该是 items[i]
36  '''

```

指定需要编辑的代码

按 "H" 查看历史输入。按 "ESC" 关闭

接受 Ctrl+Enter 拒绝 Ctrl+Backspace

```

30
31  BUGGY_CODE = '''\
32  def print_even_index_items(items):
32  def print_even_index_items(items):
33      """打印列表中所有元素的值"""
34      for i in range(len(items) + 1): # Bug: 应该是 len(items)
35          print(items[i])
36

```

```

32  def print_even_index_items(items):
33      """打印列表中所有元素的值"""
34      for i in range(len(items)):
35          print(items[i + 1]) # Bug: 应该是 items[i]

```

Questions?

课后实验

- 根据本节课的内容，借助Trae、Cursor、CodeBuddy等AI IDE构建一个自己的MCP工具（参考<https://github.com/modelcontextprotocol>使用任意语言的SDK工具构建），并接入一个LLM应用（如WorkBuddy、Claude Code）
- 参考：
https://github.com/yz1019117968/ai_coding_experiments/blob/main/mcp/code_analyzer_server.py